03_Rolling_Window_Forecast

```r
knitr::opts_chunk$set(echo = TRUE, warning = FALSE, message = FALSE)

library(knitr)
```

```
## Warning: package 'knitr' was built under R version 4.5.2
```

```r
library(tidyverse)
```

```
## Warning: package 'tidyverse' was built under R version 4.5.2

## Warning: package 'tibble' was built under R version 4.5.2

## Warning: package 'tidyr' was built under R version 4.5.2

## Warning: package 'purrr' was built under R version 4.5.2

## Warning: package 'stringr' was built under R version 4.5.2

## Warning: package 'forcats' was built under R version 4.5.2

## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.0     v readr     2.2.0
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.2     v tibble    3.3.1
## v lubridate 1.9.5     v tidyr     1.3.2
## v purrr     1.2.1
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
library(forecast)
library(lubridate)
library(ggplot2)
library(tseries)
```

```
## Registered S3 method overwritten by 'quantmod':
##   method            from
##   as.zoo.data.frame zoo
```

```r
source("../utils/one_step_model.R")
source("../utils/expanding_window.R")
```

Load Data

We use the full EDA dataset (pre-differenced) saved from notebook 01.

```r
eda_df <- readRDS("../output/eda_df_nodiff.rds")

full_df <- eda_df |> filter(date <= as.Date("2019-12-31"))

hh_full <- ts(full_df$hh_save, start = c(1977, 1), frequency = 4)
```

Rolling Window Design

Economic time series that span multiple regimes (e.g., oil shocks, the Great Moderation, the 2008 financial crisis) are prone to structural breaks. An expanding window assigns equal weight to all past observations, so data from an old regime can dilute parameter estimates for the current one. A **fixed rolling window** of size R discards observations older than R quarters, keeping estimation sample size constant and allowing the model to adapt to regime changes (Giacomini & White, 2006).

We set $R = 80$ quarters (20 years), matching the initial training length.

- Step 1: Train on 1977 Q1 – 1996 Q4 (80 obs), forecast 1997 Q1.
- Step 2: Train on 1977 Q2 – 1997 Q1 (80 obs), forecast 1997 Q2.
- Continue until the end of the series (2019 Q4).

This yields **92 one-step-ahead forecasts** (1997 Q1 – 2019 Q4), each produced by a model fitted to the most recent 80 quarters.

```
R_window <- 80  # 20 years * 4 quarters
```

```
initial_end <- as.Date("1996-12-31")

ew_train_df <- full_df |> filter(date <= initial_end)
ew_test_df  <- full_df |> filter(date > initial_end)

hh_train <- ts(ew_train_df$hh_save, start = c(1977, 1), frequency = 4)
hh_test  <- ts(ew_test_df$hh_save,  start = c(1997, 1), frequency = 4)

cat("Initial training window:", nrow(ew_train_df))
```

```
## Initial training window: 80
```

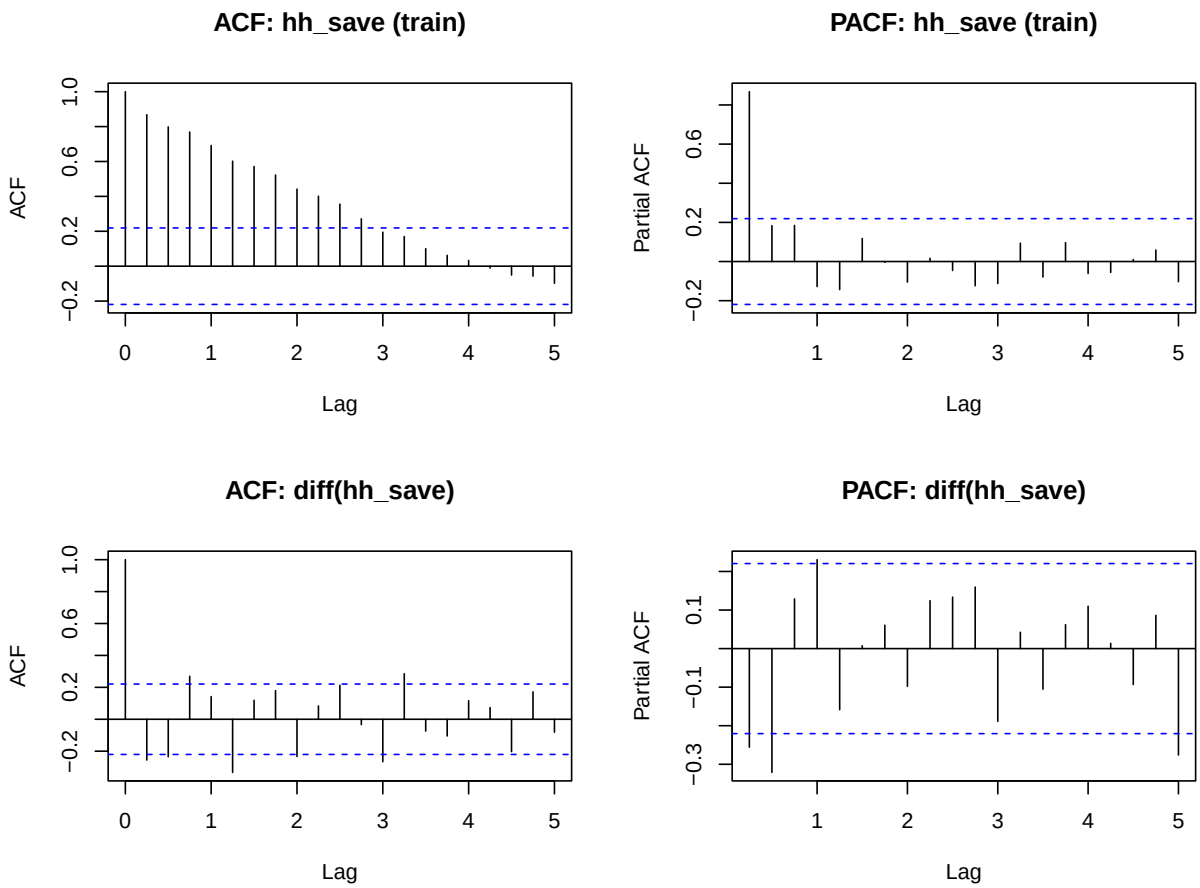
```
cat("Holdout window:", nrow(ew_test_df))
```

```
## Holdout window: 92
```

```
saveRDS(ew_train_df, file = "../output/ew_train_df.rds")
saveRDS(ew_test_df,  file = "../output/ew_test_df.rds")
```

ARIMA Order Identification

```
dhh_train <- diff(hh_train)

par(mfrow = c(2, 2))
acf(hh_train, lag.max = 20, main = "ACF: hh_save (train)")
pacf(hh_train, lag.max = 20, main = "PACF: hh_save (train)")
acf(dhh_train, lag.max = 20, main = "ACF: diff(hh_save)")
pacf(dhh_train, lag.max = 20, main = "PACF: diff(hh_save)")
```




```r
par(mfrow = c(1, 1))
```

```r
fit_auto <- auto.arima(hh_train,
                       seasonal = TRUE,
                       stepwise = FALSE,
                       approximation = FALSE)
summary(fit_auto)
```

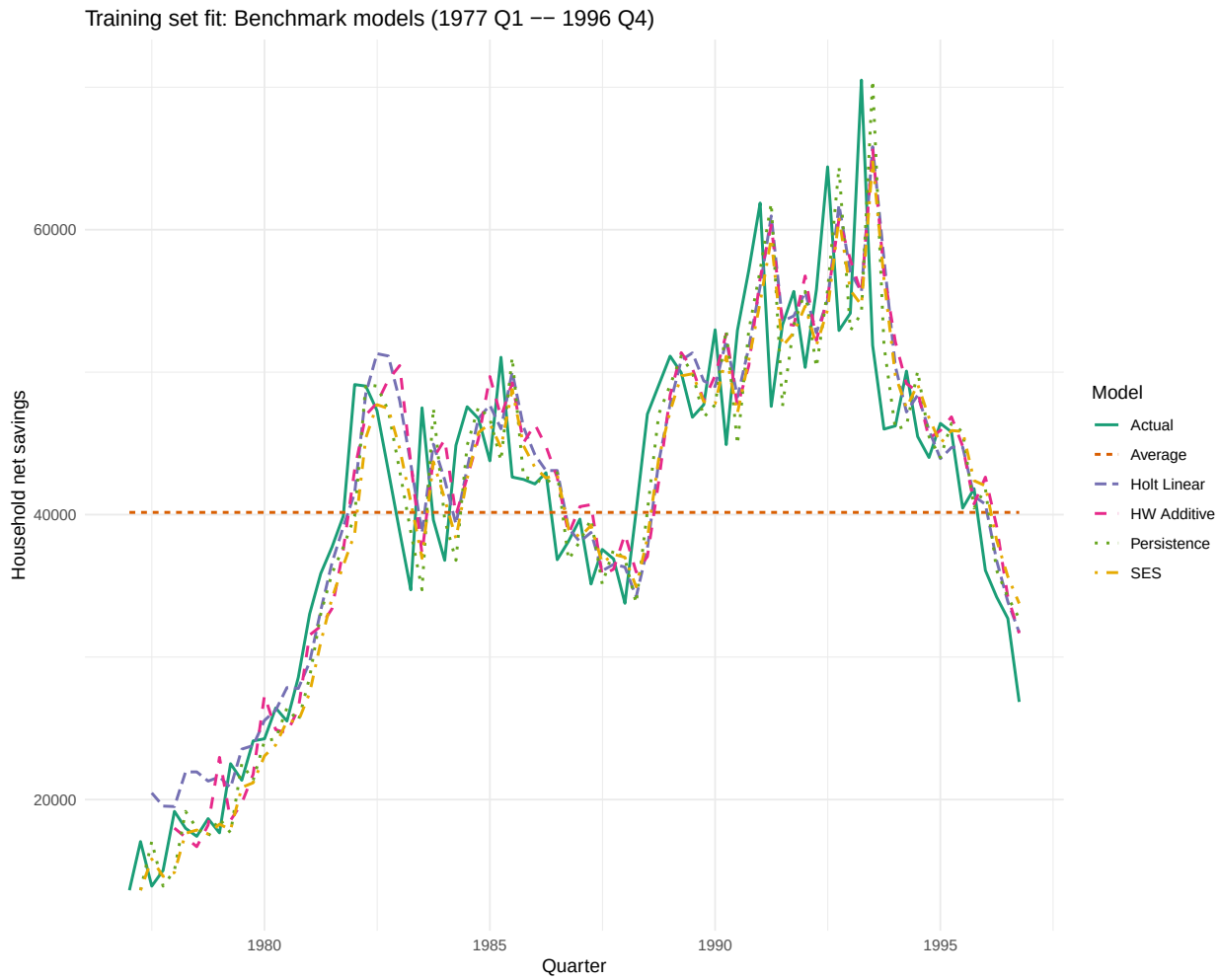
```
## Series: hh_train
## ARIMA(3,1,2)
##
## Coefficients:
##           ar1      ar2      ar3     ma1     ma2
##       -1.0731  -1.2167  -0.3103  0.8557  0.9584
## s.e.   0.1248   0.0890   0.1158  0.0859  0.1074
##
## sigma^2 = 22639578:  log likelihood = -779.99
## AIC=1571.97   AICc=1573.14   BIC=1586.19
##
## Training set error measures:
##                    ME   RMSE      MAE      MPE     MAPE      MASE        ACF1
## Training set 232.2736 4576.2 3629.904 0.429616 8.940588 0.5563834 -0.02484043
```

Training Set Fit

Before evaluating holdout performance, we inspect how each model fits the initial 80-quarter training set. This diagnostic reveals whether a model can capture the in-sample dynamics of household net savings.

Benchmark Training Fits

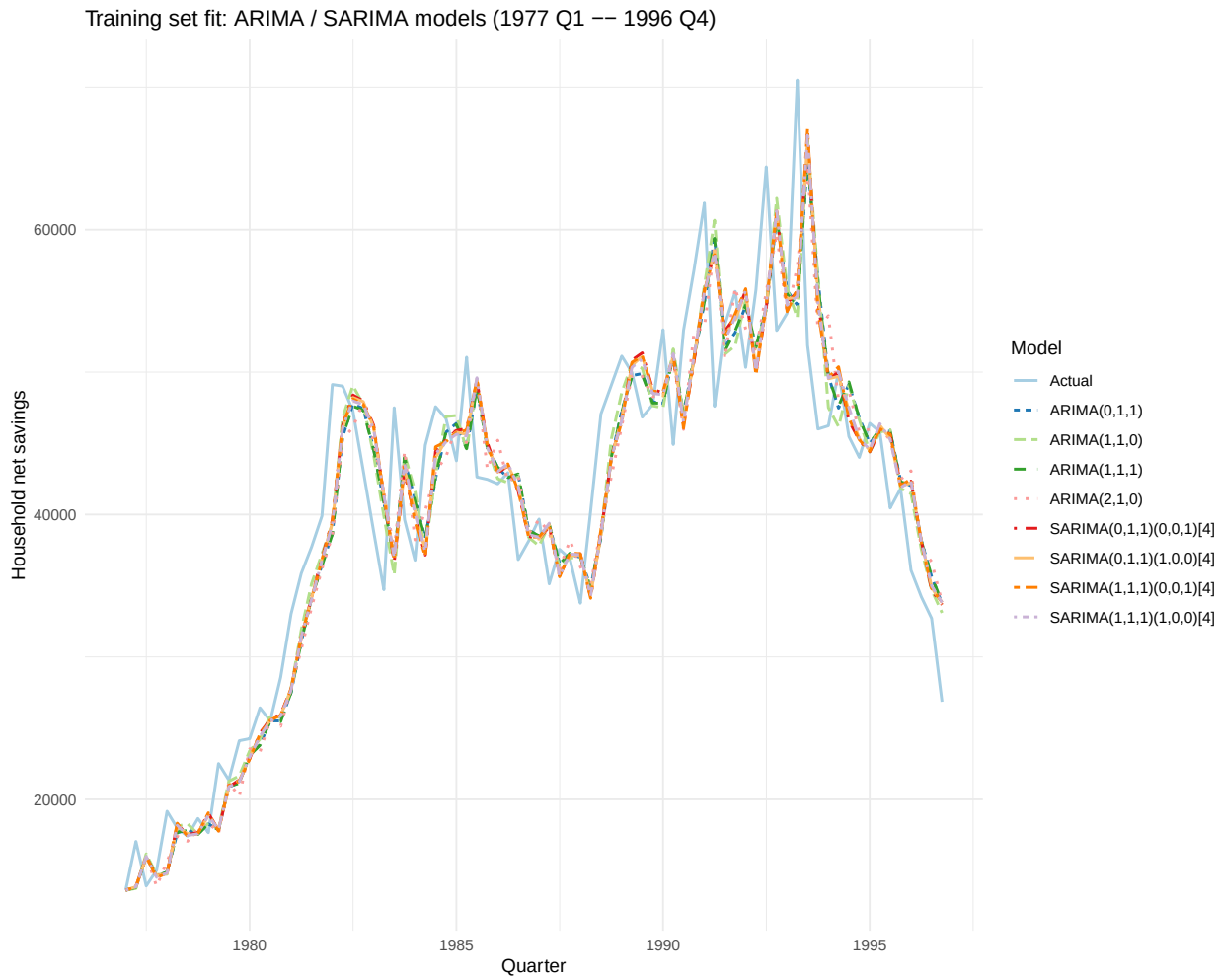
```r
n_train <- length(hh_train)
train_dates <- ew_train_df$date

# Persistence: fitted[t] = y[t-1]
persist_train_fitted <- c(NA, as.numeric(hh_train[-n_train]))

# Average: fitted[t] = overall mean of training set
avg_train_fitted <- rep(mean(hh_train), n_train)

# SES
hw_ses <- HoltWinters(hh_train, beta = FALSE, gamma = FALSE)
ses_fitted_raw <- as.numeric(hw_ses$fitted[, "xhat"])
ses_train_fitted <- c(rep(NA, n_train - length(ses_fitted_raw)), ses_fitted_raw)

# Holt Linear
hw_holt <- HoltWinters(hh_train, gamma = FALSE)
holt_fitted_raw <- as.numeric(hw_holt$fitted[, "xhat"])
holt_train_fitted <- c(rep(NA, n_train - length(holt_fitted_raw)), holt_fitted_raw)

# HW Additive
hw_add <- HoltWinters(hh_train, seasonal = "additive")
hwa_fitted_raw <- as.numeric(hw_add$fitted[, "xhat"])
hwa_train_fitted <- c(rep(NA, n_train - length(hwa_fitted_raw)), hwa_fitted_raw)

train_fit_df <- tibble(
  date = train_dates,
  Actual = as.numeric(hh_train),
  Persistence = persist_train_fitted,
  Average = avg_train_fitted,
  SES = ses_train_fitted,
  `Holt Linear` = holt_train_fitted,
  `HW Additive` = hwa_train_fitted
) |>
  pivot_longer(-date, names_to = "Model", values_to = "value")

ggplot(train_fit_df, aes(x = date, y = value, color = Model, linetype = Model)) +
  geom_line(linewidth = 0.8) +
  theme_minimal() +
  scale_color_brewer(palette = "Dark2") +
  labs(
    title = "Training set fit: Benchmark models (1977 Q1 -- 1996 Q4)",
    x = "Quarter", y = "Household net savings"
  )
```



ARIMA / SARIMA Training Fits

```
# Fit each ARIMA / SARIMA on the initial training window
arima_specs <- list(
  "ARIMA(1,1,1)" = c(1, 1, 1),
  "ARIMA(0,1,1)" = c(0, 1, 1),
  "ARIMA(2,1,0)" = c(2, 1, 0),
  "ARIMA(1,1,0)" = c(1, 1, 0)
)

sarima_specs <- list(
  "SARIMA(1,1,1)(1,0,0)[4]" = list(order = c(1,1,1), seasonal = c(1,0,0)),
  "SARIMA(0,1,1)(1,0,0)[4]" = list(order = c(0,1,1), seasonal = c(1,0,0)),
  "SARIMA(1,1,1)(0,0,1)[4]" = list(order = c(1,1,1), seasonal = c(0,0,1)),
  "SARIMA(0,1,1)(0,0,1)[4]" = list(order = c(0,1,1), seasonal = c(0,0,1))
)

# Store training fits for plotting
arima_train_fits <- list()
for (nm in names(arima_specs)) {
  fit_obj <- forecast::Arima(hh_train, order = arima_specs[[nm]], include.mean = TRUE)
```

```r
  arima_train_fits[[nm]] <- as.numeric(fitted(fit_obj))
}

sarima_train_fits <- list()
for (nm in names(sarima_specs)) {
  sp <- sarima_specs[[nm]]
  fit_obj <- forecast::Arima(hh_train, order = sp$order, seasonal = sp$seasonal,
                             include.mean = TRUE)
  sarima_train_fits[[nm]] <- as.numeric(fitted(fit_obj))
}

arima_train_df <- tibble(date = train_dates, Actual = as.numeric(hh_train))
for (nm in names(arima_train_fits)) {
  arima_train_df[[nm]] <- arima_train_fits[[nm]]
}
for (nm in names(sarima_train_fits)) {
  arima_train_df[[nm]] <- sarima_train_fits[[nm]]
}

arima_train_long <- arima_train_df |>
  pivot_longer(-date, names_to = "Model", values_to = "value")

ggplot(arima_train_long, aes(x = date, y = value, color = Model, linetype = Model)) +
  geom_line(linewidth = 0.8) +
  theme_minimal() +
  scale_color_brewer(palette = "Paired") +
  labs(
    title = "Training set fit: ARIMA / SARIMA models (1977 Q1 -- 1996 Q4)",
    x = "Quarter", y = "Household net savings"
  )
```




```r
train_rmse <- function(actual, fitted) {
  valid <- !is.na(fitted)
  sqrt(mean((actual[valid] - fitted[valid])^2))
}

train_rmse_tbl <- tibble(
  Model = c("Persistence", "Average", "SES", "Holt Linear", "HW Additive",
            names(arima_train_fits), names(sarima_train_fits)),
  Train_RMSE = round(c(
    train_rmse(as.numeric(hh_train), persist_train_fitted),
    train_rmse(as.numeric(hh_train), avg_train_fitted),
    train_rmse(as.numeric(hh_train), ses_train_fitted),
    train_rmse(as.numeric(hh_train), holt_train_fitted),
    train_rmse(as.numeric(hh_train), hwa_train_fitted),
    sapply(arima_train_fits, \(f) train_rmse(as.numeric(hh_train), f)),
    sapply(sarima_train_fits, \(f) train_rmse(as.numeric(hh_train), f))
  ), 4)
) |>
  arrange(Train_RMSE)

# Print training RMSE table
train_rmse_tbl
```

```
## # A tibble: 13 x 2
##    Model                    Train_RMSE
##    <chr>                         <dbl>
##  1 ARIMA(2,1,0)                  5005.
##  2 SARIMA(1,1,1)(0,0,1)[4]       5109.
##  3 SARIMA(0,1,1)(0,0,1)[4]       5114.
##  4 SARIMA(1,1,1)(1,0,0)[4]       5126.
##  5 SARIMA(0,1,1)(1,0,0)[4]       5130.
##  6 ARIMA(1,1,1)                  5181.
##  7 ARIMA(0,1,1)                  5183.
##  8 SES                           5217.
##  9 ARIMA(1,1,0)                  5290.
## 10 Holt Linear                   5353.
## 11 Persistence                   5508.
## 12 HW Additive                   5508.
## 13 Average                      12450.
```

Benchmark Holdout Forecasts

We evaluate all benchmark rules on the 92-quarter holdout for reference.

```r
ew_rmse_tbl <- tibble(Model = character(), Method = character(), RMSE = double())
ew_fc_store <- list()

# Persistence
res <- persist_fc(hh_train, hh_test)
ew_rmse_tbl <- add_row(ew_rmse_tbl, Model = "Persistence", Method = "Benchmark", RMSE = res$rmse)
ew_fc_store$Persistence <- res$fc

# Average
res <- iid_fc(hh_train, hh_test)
ew_rmse_tbl <- add_row(ew_rmse_tbl, Model = "Average", Method = "Benchmark", RMSE = res$rmse)
ew_fc_store$Average <- res$fc

# Average by Period
res <- iidbyperiod_fc(hh_train, hh_test, periodicity = 4)
ew_rmse_tbl <- add_row(ew_rmse_tbl, Model = "Average by Period", Method = "Benchmark", RMSE = res$rmse)
ew_fc_store$`Average by Period` <- res$fc

# SES
res <- esm_fc(hh_train, hh_test,
              alpha = hw_ses$alpha,
              level = as.numeric(hw_ses$coefficients[1]))
ew_rmse_tbl <- add_row(ew_rmse_tbl, Model = "SES", Method = "Benchmark", RMSE = res$rmse)
ew_fc_store$SES <- res$fc

# Holt Linear
res <- lholt_fc(hh_train, hh_test,
                alpha = hw_holt$alpha,
                beta  = hw_holt$beta,
                level = as.numeric(hw_holt$coefficients[1]),
                slope = as.numeric(hw_holt$coefficients[2]))
ew_rmse_tbl <- add_row(ew_rmse_tbl, Model = "Holt Linear", Method = "Benchmark", RMSE = res$rmse)
ew_fc_store$`Holt Linear` <- res$fc
```

```r
# HW Additive
res <- aseason_fc(hh_train, hh_test,
                  alpha  = hw_add$alpha,
                  beta   = hw_add$beta,
                  gamma  = hw_add$gamma,
                  level  = as.numeric(hw_add$coefficients[1]),
                  slope  = as.numeric(hw_add$coefficients[2]),
                  season = as.numeric(hw_add$coefficients[3:6]))
ew_rmse_tbl <- add_row(ew_rmse_tbl, Model = "HW Additive", Method = "Benchmark", RMSE = res$rmse)
ew_fc_store$`HW Additive` <- res$fc

ew_rmse_tbl
```

```
## # A tibble: 6 x 3
##   Model             Method      RMSE
##   <chr>             <chr>      <dbl>
## 1 Persistence       Benchmark  9032.
## 2 Average           Benchmark 18189.
## 3 Average by Period Benchmark 18230.
## 4 SES               Benchmark  8907.
## 5 Holt Linear       Benchmark  9345.
## 6 HW Additive       Benchmark  9464.
```

Fixed-Parameter One-Step Forecasts (ARIMA / SARIMA)

For reference, we run fixed-parameter one-step-ahead forecasts on this longer holdout.

```r
arima_fixed_results <- list()

for (nm in names(arima_specs)) {
  res <- arima_one_step_fc(hh_train, hh_test,
                           order = arima_specs[[nm]],
                           include.mean = TRUE)
  arima_fixed_results[[nm]] <- res
  ew_rmse_tbl <- add_row(ew_rmse_tbl, Model = nm, Method = "Fixed", RMSE = res$rmse)
  ew_fc_store[[paste0(nm, " (fixed)")]] <- res$fc
}

arima_fixed_tbl <- tibble(
  Model = names(arima_specs),
  AIC = sapply(arima_fixed_results, \(r) round(r$fit$aic, 2)),
  RMSE_Fixed = sapply(arima_fixed_results, \(r) round(r$rmse, 4))
)
arima_fixed_tbl
```

```
## # A tibble: 4 x 3
##   Model          AIC RMSE_Fixed
##   <chr>        <dbl>      <dbl>
## 1 ARIMA(1,1,1) 1583.      8865.
## 2 ARIMA(0,1,1) 1581.      8905.
## 3 ARIMA(2,1,0) 1577.      9215.
## 4 ARIMA(1,1,0) 1584.      8939.
```

```r
sarima_fixed_results <- list()
for (nm in names(sarima_specs)) {
```

```r
  sp <- sarima_specs[[nm]]
  res <- sarima_one_step_fc(hh_train, hh_test,
                            order = sp$order,
                            seasonal = sp$seasonal,
                            include.mean = TRUE)
  sarima_fixed_results[[nm]] <- res
  ew_rmse_tbl <- add_row(ew_rmse_tbl, Model = nm, Method = "Fixed", RMSE = res$rmse)
  ew_fc_store[[paste0(nm, " (fixed)")]] <- res$fc
}

sarima_fixed_tbl <- tibble(
  Model = names(sarima_specs),
  AIC = sapply(sarima_fixed_results, \(r) round(r$fit$aic, 2)),
  RMSE_Fixed = sapply(sarima_fixed_results, \(r) round(r$rmse, 4))
)
sarima_fixed_tbl
```

```
## # A tibble: 4 x 3
##   Model                     AIC RMSE_Fixed
##   <chr>                   <dbl>      <dbl>
## 1 SARIMA(1,1,1)(1,0,0)[4] 1583.      8941.
## 2 SARIMA(0,1,1)(1,0,0)[4] 1581.      9000.
## 3 SARIMA(1,1,1)(0,0,1)[4] 1583.      8970.
## 4 SARIMA(0,1,1)(0,0,1)[4] 1581.      9034.
```

Rolling Window Forecasts

Now we re-estimate each model at every holdout step.

```r
arima_ew_results <- list()
for (nm in names(arima_specs)) {
  res <- arima_expanding_window_fc(hh_train, hh_test,
                                   order = arima_specs[[nm]],
                                   include.mean = TRUE,
                                   window_size = R_window)
  arima_ew_results[[nm]] <- res
  ew_rmse_tbl <- add_row(ew_rmse_tbl, Model = nm, Method = "Rolling",
                         RMSE = res$rmse)
  ew_fc_store[[paste0(nm, " (rolling)")]] <- res$fc
}
```

```r
sarima_ew_results <- list()
for (nm in names(sarima_specs)) {
  sp <- sarima_specs[[nm]]
  res <- sarima_expanding_window_fc(hh_train, hh_test,
                                    order = sp$order,
                                    seasonal = sp$seasonal,
                                    include.mean = TRUE,
                                    window_size = R_window)
  sarima_ew_results[[nm]] <- res
  ew_rmse_tbl <- add_row(ew_rmse_tbl, Model = nm, Method = "Rolling",
                         RMSE = res$rmse)
  ew_fc_store[[paste0(nm, " (rolling)")]] <- res$fc
}
```

ARIMAX Rolling Window Forecast

Variable Selection and Feasibility

We use the same CCF-justified regressors as in notebook 02: lagged log-differences of GPDI and CPI (`ldgpdi_lag1`, `ldcpi_lag2`, `ldgpdi_lag3`). These were selected based on significant cross-correlations at negative lags (the regressor leads household savings).

Feasibility of variable selection in the rolling window:

- The CCF-based selection was originally performed on the notebook 02 training sample. For the rolling window we apply the same regressor set starting from the 1977–1996 initial window. This is conservative: if the leading relationship holds over the longer sample, it should manifest in sub-windows as well.
- Lagged regressors (`lag 1`, `lag 2`, `lag 3`) are available at forecast time by construction, so there is no look-ahead bias.
- The rolling window lets us empirically verify whether the regressor–response relationship persists as the training window rolls through different regimes.

```r
# Build ARIMAX regressors from the full pre-2020 dataset
arimax_ew_df <- full_df |>
  arrange(date) |>
  mutate(
    ldgpdi      = log(gpdi) - log(lag(gpdi)),
    ldcpi       = log1p(cpi) - log1p(lag(cpi)),
    ldcpi_lag2  = lag(ldcpi, 2),
    ldgpdi_lag1 = lag(ldgpdi, 1),
    ldgpdi_lag3 = lag(ldgpdi, 3)
  ) |>
  drop_na(ldgpdi_lag1, ldcpi_lag2, ldgpdi_lag3)

arimax_vars <- c("ldgpdi_lag1", "ldcpi_lag2", "ldgpdi_lag3")
arimax_reduced_vars <- c("ldgpdi_lag1")

# Split at the initial training boundary - full regressors
x_ew_train <- arimax_ew_df |> filter(date <= initial_end) |>
  select(all_of(arimax_vars)) |> as.matrix()
x_ew_test <- arimax_ew_df |> filter(date > initial_end) |>
  select(all_of(arimax_vars)) |> as.matrix()

# Reduced regressors (only ldgpdi_lag1)
x_ew_train_r <- arimax_ew_df |> filter(date <= initial_end) |>
  select(all_of(arimax_reduced_vars)) |> as.matrix()
x_ew_test_r <- arimax_ew_df |> filter(date > initial_end) |>
  select(all_of(arimax_reduced_vars)) |> as.matrix()

# Align hh_save to the ARIMAX-available dates (loses ~4 initial obs due to lagging)
hh_train_x <- ts(
  arimax_ew_df |> filter(date <= initial_end) |> pull(hh_save),
  start = c(year(min(arimax_ew_df$date)), quarter(min(arimax_ew_df$date))),
  frequency = 4
)
hh_test_x <- ts(
  arimax_ew_df |> filter(date > initial_end) |> pull(hh_save),
  start = c(
    year(min(arimax_ew_df |> filter(date > initial_end) |> pull(date))),
    quarter(min(arimax_ew_df |> filter(date > initial_end) |> pull(date)))
```

```r
  ),
  frequency = 4
)

cat("ARIMAX initial training window:", length(hh_train_x))
```

```
## ARIMAX initial training window: 76
```

```r
cat("ARIMAX holdout window:", length(hh_test_x))
```

```
## ARIMAX holdout window: 92
```

ARIMAX Coefficient Significance (Initial Window)

We fit ARIMAX(1,1,1) on the initial 1977–1996 window to inspect whether the exogenous coefficients are significant before committing to the full rolling window evaluation.

```r
arimax_specs <- list(
  "ARIMAX(1,1,1) (Full)" = c(1, 1, 1),
  "ARIMAX(0,1,1) (Full)" = c(0, 1, 1),
  "ARIMAX(1,1,2) (Full)" = c(1, 1, 2),
  "ARIMAX(2,1,2) (Full)" = c(2, 1, 2)
)

arimax_reduced_specs <- list(
  "ARIMAX(1,1,1) (Reduced)" = c(1, 1, 1),
  "ARIMAX(0,1,1) (Reduced)" = c(0, 1, 1),
  "ARIMAX(1,1,2) (Reduced)" = c(1, 1, 2),
  "ARIMAX(2,1,2) (Reduced)" = c(2, 1, 2)
)

fit_arimax_init <- forecast::Arima(hh_train_x, order = c(1, 1, 1),
                                   xreg = x_ew_train, include.mean = TRUE)
summary(fit_arimax_init)
```

```
## Series: hh_train_x
## Regression with ARIMA(1,1,1) errors
##
## Coefficients:
##          ar1      ma1  ldgpdi_lag1  ldcpi_lag2  ldgpdi_lag3
##       0.1196  -0.4335    -22186.58    49475.78     20658.80
## s.e.  0.2407   0.1990     10879.07   101999.38     11292.85
##
## sigma^2 = 27065690:  log likelihood = -745.66
## AIC=1503.32   AICc=1504.55   BIC=1517.22
##
## Training set error measures:
##                    ME     RMSE      MAE       MPE     MAPE      MASE
## Training set 256.9954 4992.887 3845.361 0.1431084 8.901065 0.5741392
##                     ACF1
## Training set 0.006970882
```

```r
# Extract coefficient significance
coefs <- coef(fit_arimax_init)
ses   <- sqrt(diag(vcov(fit_arimax_init)))
z_vals <- coefs / ses
```

```r
p_vals <- 2 * (1 - pnorm(abs(z_vals)))

coef_tbl <- tibble(
  Coefficient = names(coefs),
  Estimate = round(coefs, 6),
  SE = round(ses, 6),
  z = round(z_vals, 3),
  p_value = round(p_vals, 4),
  Significant = ifelse(p_vals < 0.05, "Yes", "No")
)
coef_tbl
```

```
## # A tibble: 5 x 6
##   Coefficient   Estimate         SE      z p_value Significant
##   <chr>            <dbl>      <dbl>  <dbl>   <dbl> <chr>
## 1 ar1              0.120      0.241  0.497  0.619  No
## 2 ma1             -0.434      0.199 -2.18   0.0293 Yes
## 3 ldgpdi_lag1 -22187.     10879.    -2.04   0.0414 Yes
## 4 ldcpi_lag2   49476.    101999.     0.485  0.628  No
## 5 ldgpdi_lag3  20659.     11293.     1.83   0.0673 No
```

ARIMAX Fixed-Parameter Forecasts (Full Regressors)

```r
arimax_fixed_results <- list()
for (nm in names(arimax_specs)) {
  res <- arimax_one_step_fc(hh_train_x, hh_test_x,
                            xreg_train = x_ew_train,
                            xreg_holdout = x_ew_test,
                            order = arimax_specs[[nm]],
                            include.mean = TRUE)
  arimax_fixed_results[[nm]] <- res
  ew_rmse_tbl <- add_row(ew_rmse_tbl, Model = nm, Method = "Fixed", RMSE = res$rmse)
  ew_fc_store[[paste0(nm, " (fixed)")]] <- res$fc
}

arimax_fixed_tbl <- tibble(
  Model = names(arimax_specs),
  AIC = sapply(arimax_fixed_results, \(r) round(r$fit$aic, 2)),
  RMSE_Fixed = sapply(arimax_fixed_results, \(r) round(r$rmse, 4))
)
arimax_fixed_tbl
```

```
## # A tibble: 4 x 3
##   Model                  AIC RMSE_Fixed
##   <chr>                <dbl>      <dbl>
## 1 ARIMAX(1,1,1) (Full) 1503.      8819.
## 2 ARIMAX(0,1,1) (Full) 1502.      8888.
## 3 ARIMAX(1,1,2) (Full) 1504.      8889.
## 4 ARIMAX(2,1,2) (Full) 1501.      9478.
```

ARIMAX Rolling Window Forecasts (Full Regressors)

```r
arimax_ew_results <- list()
for (nm in names(arimax_specs)) {
```

```r
  res <- arimax_expanding_window_fc(hh_train_x, hh_test_x,
                                    xreg_train = x_ew_train,
                                    xreg_holdout = x_ew_test,
                                    order = arimax_specs[[nm]],
                                    include.mean = TRUE,
                                    window_size = R_window)
  arimax_ew_results[[nm]] <- res
  ew_rmse_tbl <- add_row(ew_rmse_tbl, Model = nm, Method = "Rolling",
                         RMSE = res$rmse)
  ew_fc_store[[paste0(nm, " (rolling)")]] <- res$fc
}
```

ARIMAX Fixed-Parameter Forecasts (Reduced: `ldgpdi_lag1` only)

```r
# Re-run fixed-parameter forecasts with only the significant regressor (ldgpdi_lag1)
arimax_r_fixed_results <- list()
for (nm in names(arimax_reduced_specs)) {
  res <- arimax_one_step_fc(hh_train_x, hh_test_x,
                            xreg_train = x_ew_train_r,
                            xreg_holdout = x_ew_test_r,
                            order = arimax_reduced_specs[[nm]],
                            include.mean = TRUE)
  arimax_r_fixed_results[[nm]] <- res
  ew_rmse_tbl <- add_row(ew_rmse_tbl, Model = nm, Method = "Fixed", RMSE = res$rmse)
  ew_fc_store[[paste0(nm, " (fixed)")]] <- res$fc
}

arimax_r_fixed_tbl <- tibble(
  Model = names(arimax_reduced_specs),
  AIC = sapply(arimax_r_fixed_results, \(r) round(r$fit$aic, 2)),
  RMSE_Fixed = sapply(arimax_r_fixed_results, \(r) round(r$rmse, 4))
)
arimax_r_fixed_tbl
```

```
## # A tibble: 4 x 3
##   Model                     AIC RMSE_Fixed
##   <chr>                   <dbl>      <dbl>
## 1 ARIMAX(1,1,1) (Reduced) 1503.      8824.
## 2 ARIMAX(0,1,1) (Reduced) 1501.      8841.
## 3 ARIMAX(1,1,2) (Reduced) 1504.      8789.
## 4 ARIMAX(2,1,2) (Reduced) 1498.      9691.
```

ARIMAX Rolling Window Forecasts (Reduced: `ldgpdi_lag1` only)

```r
arimax_r_ew_results <- list()
for (nm in names(arimax_reduced_specs)) {
  res <- arimax_expanding_window_fc(hh_train_x, hh_test_x,
                                    xreg_train = x_ew_train_r,
                                    xreg_holdout = x_ew_test_r,
                                    order = arimax_reduced_specs[[nm]],
                                    include.mean = TRUE,
                                    window_size = R_window)
  arimax_r_ew_results[[nm]] <- res
```

```r
  ew_rmse_tbl <- add_row(ew_rmse_tbl, Model = nm, Method = "Rolling",
                         RMSE = res$rmse)
  ew_fc_store[[paste0(nm, " (rolling)")]] <- res$fc
}
```

Comparison Tables

Fixed vs Rolling Window

```r
# ARIMA: fixed vs rolling
arima_compare <- tibble(
  Model = names(arima_specs),
  RMSE_Fixed = sapply(arima_fixed_results, \(r) round(r$rmse, 4)),
  RMSE_Rolling = sapply(arima_ew_results, \(r) round(r$rmse, 4)),
  Improvement_pct = round((RMSE_Fixed - RMSE_Rolling) / RMSE_Fixed * 100, 2)
)

arima_compare
```

```
## # A tibble: 4 x 4
##   Model        RMSE_Fixed RMSE_Rolling Improvement_pct
##   <chr>             <dbl>        <dbl>           <dbl>
## 1 ARIMA(1,1,1)      8865.        8914.           -0.54
## 2 ARIMA(0,1,1)      8905.        9008.           -1.17
## 3 ARIMA(2,1,0)      9215.        9122.            1.01
## 4 ARIMA(1,1,0)      8939.        8979.           -0.45
```

```r
# SARIMA: fixed vs rolling
sarima_compare <- tibble(
  Model = names(sarima_specs),
  RMSE_Fixed = sapply(sarima_fixed_results, \(r) round(r$rmse, 4)),
  RMSE_Rolling = sapply(sarima_ew_results, \(r) round(r$rmse, 4)),
  Improvement_pct = round((RMSE_Fixed - RMSE_Rolling) / RMSE_Fixed * 100, 2)
)

sarima_compare
```

```
## # A tibble: 4 x 4
##   Model                   RMSE_Fixed RMSE_Rolling Improvement_pct
##   <chr>                        <dbl>        <dbl>           <dbl>
## 1 SARIMA(1,1,1)(1,0,0)[4]      8941.        8956.           -0.17
## 2 SARIMA(0,1,1)(1,0,0)[4]      9000.        9196.           -2.17
## 3 SARIMA(1,1,1)(0,0,1)[4]      8970.        9031.           -0.68
## 4 SARIMA(0,1,1)(0,0,1)[4]      9034.        9227.           -2.13
```

```r
# ARIMAX (Full): fixed vs rolling
arimax_compare <- tibble(
  Model = names(arimax_specs),
  RMSE_Fixed = sapply(arimax_fixed_results, \(r) round(r$rmse, 4)),
  RMSE_Rolling = sapply(arimax_ew_results, \(r) round(r$rmse, 4)),
  Improvement_pct = round((RMSE_Fixed - RMSE_Rolling) / RMSE_Fixed * 100, 2)
)
arimax_compare
```

```
## # A tibble: 4 x 4
```

```
##   Model                 RMSE_Fixed RMSE_Rolling Improvement_pct
##   <chr>                      <dbl>        <dbl>           <dbl>
## 1 ARIMAX(1,1,1) (Full)      8819.        8929.           -1.24
## 2 ARIMAX(0,1,1) (Full)      8888.        9140.           -2.84
## 3 ARIMAX(1,1,2) (Full)      8889.        9070.           -2.03
## 4 ARIMAX(2,1,2) (Full)      9478.        9035.            4.68
```

```r
# ARIMAX (Reduced): fixed vs rolling
arimax_r_compare <- tibble(
  Model = names(arimax_reduced_specs),
  RMSE_Fixed = sapply(arimax_r_fixed_results, \(r) round(r$rmse, 4)),
  RMSE_Rolling = sapply(arimax_r_ew_results, \(r) round(r$rmse, 4)),
  Improvement_pct = round((RMSE_Fixed - RMSE_Rolling) / RMSE_Fixed * 100, 2)
)
arimax_r_compare
```

```
## # A tibble: 4 x 4
##   Model                    RMSE_Fixed RMSE_Rolling Improvement_pct
##   <chr>                         <dbl>        <dbl>           <dbl>
## 1 ARIMAX(1,1,1) (Reduced)      8824.        8740.            0.96
## 2 ARIMAX(0,1,1) (Reduced)      8841.        8941.           -1.13
## 3 ARIMAX(1,1,2) (Reduced)      8789.        8803.           -0.15
## 4 ARIMAX(2,1,2) (Reduced)      9691.        9150.            5.58
```

Full RMSE Ranking (Benchmarks + ARIMA/SARIMA/ARIMAX)

```r
ew_rmse_tbl <- ew_rmse_tbl |>
  mutate(RMSE = round(RMSE, 4)) |>
  arrange(RMSE)

ew_rmse_tbl
```

```
## # A tibble: 38 x 3
##    Model                   Method   RMSE
##    <chr>                   <chr>   <dbl>
##  1 ARIMAX(1,1,1) (Reduced) Rolling 8740.
##  2 ARIMAX(1,1,2) (Reduced) Fixed   8789.
##  3 ARIMAX(1,1,2) (Reduced) Rolling 8803.
##  4 ARIMAX(1,1,1) (Full)    Fixed   8819.
##  5 ARIMAX(1,1,1) (Reduced) Fixed   8824.
##  6 ARIMAX(0,1,1) (Reduced) Fixed   8841.
##  7 ARIMA(1,1,1)            Fixed   8865.
##  8 ARIMAX(0,1,1) (Full)    Fixed   8888.
##  9 ARIMAX(1,1,2) (Full)    Fixed   8889.
## 10 ARIMA(0,1,1)            Fixed   8905.
## # i 28 more rows
```

```r
best_ew_model <- ew_rmse_tbl |> slice(1)

best_ew_model
```

```
## # A tibble: 1 x 3
##   Model                   Method   RMSE
##   <chr>                   <chr>   <dbl>
## 1 ARIMAX(1,1,1) (Reduced) Rolling 8740.
```

```r
# How do rolling window models compare to persistence?
persist_rmse <- ew_rmse_tbl |>
  filter(Model == "Persistence") |>
  pull(RMSE)


# Print models that beat persistence
ew_rmse_tbl |>
  filter(RMSE < persist_rmse & Method == "Rolling")
```

```
## # A tibble: 9 x 3
##   Model                    Method   RMSE
##   <chr>                    <chr>   <dbl>
## 1 ARIMAX(1,1,1) (Reduced) Rolling 8740.
## 2 ARIMAX(1,1,2) (Reduced) Rolling 8803.
## 3 ARIMA(1,1,1)            Rolling 8914.
## 4 ARIMAX(1,1,1) (Full)    Rolling 8929.
## 5 ARIMAX(0,1,1) (Reduced) Rolling 8941.
## 6 SARIMA(1,1,1)(1,0,0)[4] Rolling 8956.
## 7 ARIMA(1,1,0)            Rolling 8979.
## 8 ARIMA(0,1,1)            Rolling 9008.
## 9 SARIMA(1,1,1)(0,0,1)[4] Rolling 9031.
```

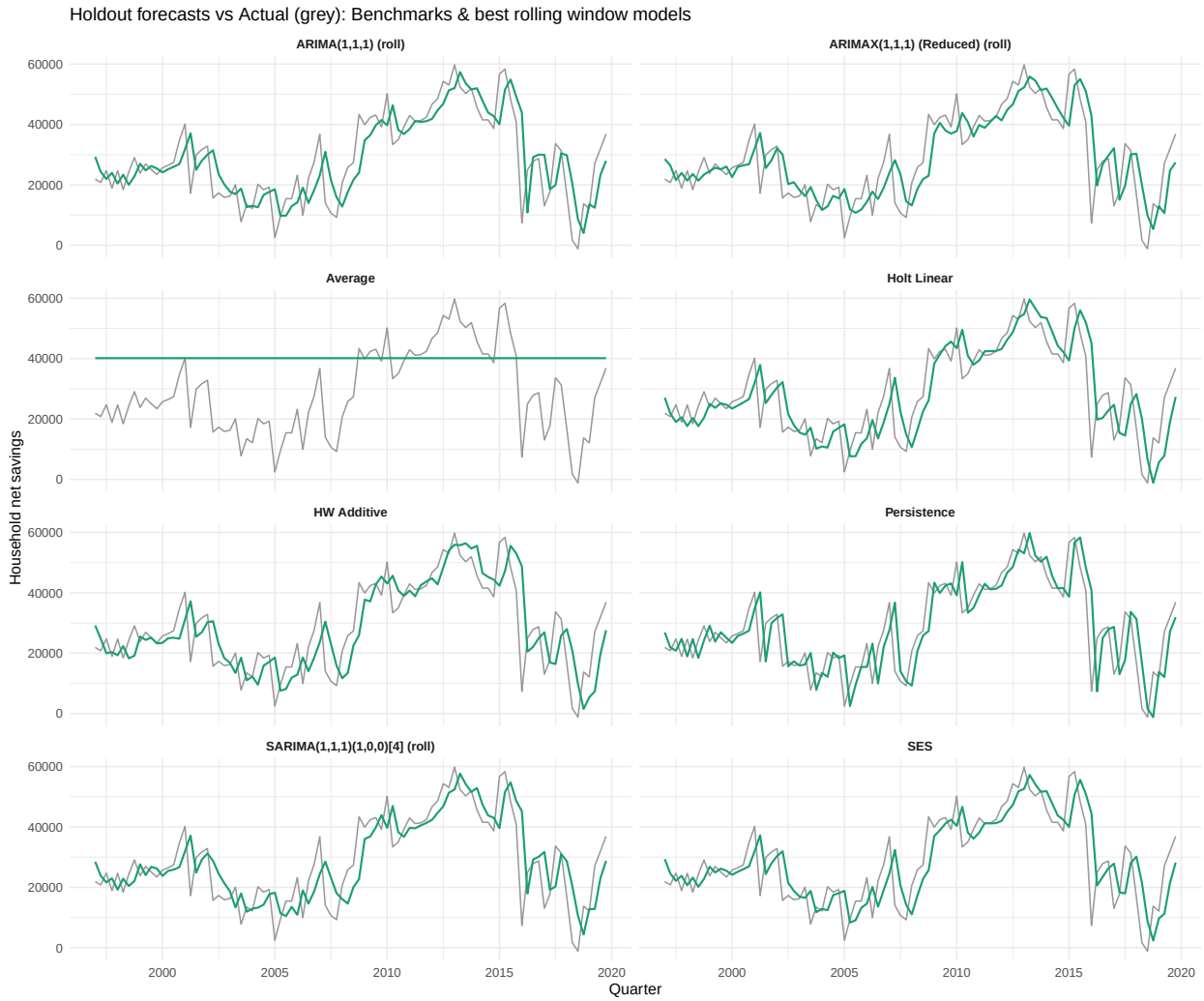
```r
saveRDS(ew_rmse_tbl, file = "../output/ew_rmse_tbl.rds")
```

Holdout Visualization

Benchmarks vs Best Rolling Window Models

```r
best_arima_nm  <- arima_compare$Model[which.min(arima_compare$RMSE_Rolling)]
best_sarima_nm <- sarima_compare$Model[which.min(sarima_compare$RMSE_Rolling)]
arimax_all_compare <- bind_rows(arimax_compare, arimax_r_compare)
best_arimax_nm <- arimax_all_compare$Model[which.min(arimax_all_compare$RMSE_Rolling)]

fc_all_df <- tibble(
  date = ew_test_df$date,
  Actual = as.numeric(hh_test),
  Persistence = as.numeric(ew_fc_store$Persistence),
  Average = as.numeric(ew_fc_store$Average),
  SES = as.numeric(ew_fc_store$SES),
  `Holt Linear` = as.numeric(ew_fc_store$`Holt Linear`),
  `HW Additive` = as.numeric(ew_fc_store$`HW Additive`)
)
fc_all_df[[paste0(best_arima_nm, " (roll)")]] <-
  as.numeric(ew_fc_store[[paste0(best_arima_nm, " (rolling)")]])
fc_all_df[[paste0(best_sarima_nm, " (roll)")]] <-
  as.numeric(ew_fc_store[[paste0(best_sarima_nm, " (rolling)")]])
fc_all_df[[paste0(best_arimax_nm, " (roll)")]] <-
  as.numeric(ew_fc_store[[paste0(best_arimax_nm, " (rolling)")]])

fc_all_long <- fc_all_df |>
  pivot_longer(-date, names_to = "Model", values_to = "value")

# Faceted panel: each model vs Actual (grey)
```

```r
actual_df <- fc_all_long |> filter(Model == "Actual")
model_df  <- fc_all_long |> filter(Model != "Actual")

ggplot(model_df, aes(x = date)) +
  geom_line(data = actual_df |> select(-Model) |>
              crossing(Model = unique(model_df$Model)),
            aes(y = value), color = "grey60", linewidth = 0.5) +
  geom_line(aes(y = value), color = "#1b9e77", linewidth = 0.7) +
  facet_wrap(~ Model, ncol = 2) +
  theme_minimal(base_size = 11) +
  theme(strip.text = element_text(face = "bold")) +
  labs(
    title = "Holdout forecasts vs Actual (grey): Benchmarks & best rolling window models",
    x = "Quarter", y = "Household net savings"
  )
```

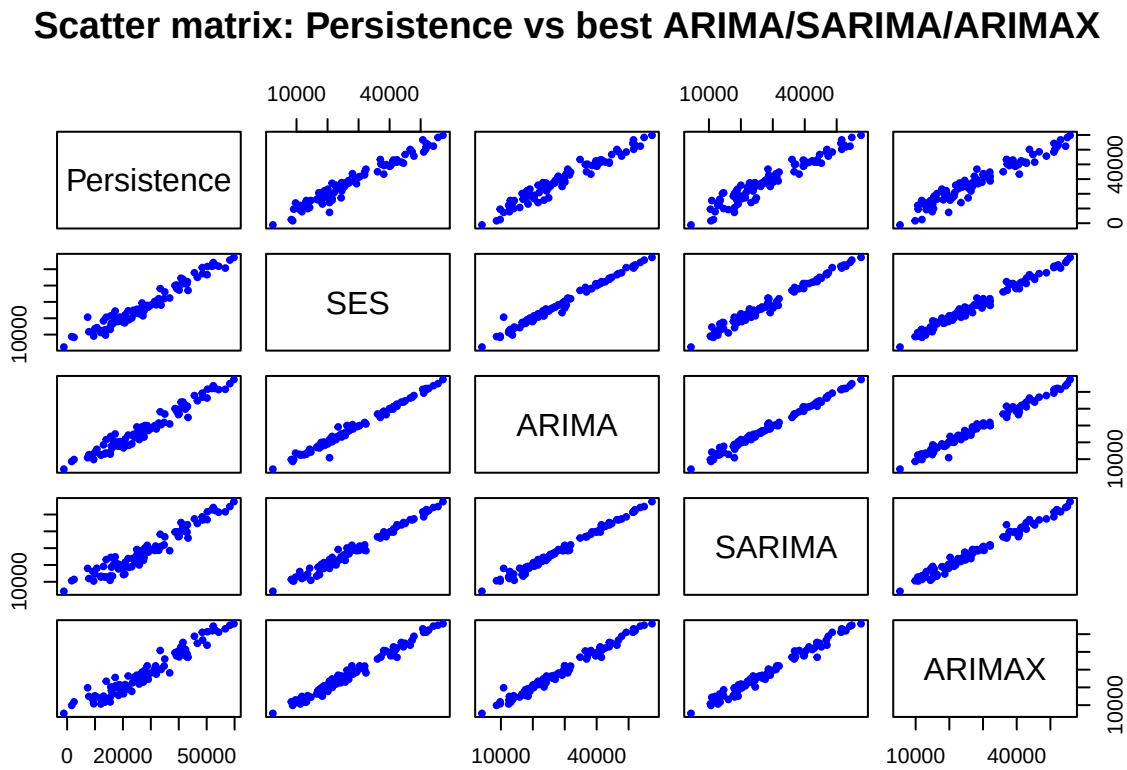


Fixed vs Rolling: Best ARIMA, SARIMA, ARIMAX

```r
# Build a combined faceted comparison: Fixed vs Rolling for best of each family
compare_models <- list(
```

```r
  ARIMA  = best_arima_nm,
  SARIMA = best_sarima_nm,
  ARIMAX = best_arimax_nm
)

compare_df <- tibble(date = ew_test_df$date, Actual = as.numeric(hh_test))
compare_long_list <- list()

for (family in names(compare_models)) {
  nm <- compare_models[[family]]
  panel_df <- tibble(
    date = ew_test_df$date,
    Actual        = as.numeric(hh_test),
    `Fixed params` = as.numeric(ew_fc_store[[paste0(nm, " (fixed)")]]),
    `Rolling window` = as.numeric(ew_fc_store[[paste0(nm, " (rolling)")]]),
    Persistence   = as.numeric(ew_fc_store$Persistence)
  ) |>
    pivot_longer(-date, names_to = "Method", values_to = "value") |>
    mutate(Panel = paste0(family, ": ", nm))
  compare_long_list[[family]] <- panel_df
}

compare_long <- bind_rows(compare_long_list)

ggplot(compare_long, aes(x = date, y = value, color = Method, linetype = Method)) +
  geom_line(linewidth = 0.8) +
  facet_wrap(~ Panel, ncol = 1, scales = "free_y") +
  theme_minimal(base_size = 11) +
  theme(strip.text = element_text(face = "bold"),
        legend.position = "bottom") +
  scale_color_brewer(palette = "Dark2") +
  labs(
    title = "Fixed vs Rolling Window: Best ARIMA, SARIMA, ARIMAX",
    x = "Quarter", y = "Household net savings"
  )
```

```r
# Scatter matrix of forecasts: Persistence vs HW vs best ARIMA/SARIMA/ARIMAX
scatter_df <- tibble(
  Persistence = as.numeric(ew_fc_store$Persistence),
  SES = as.numeric(ew_fc_store$`SES`),
  ARIMA = as.numeric(ew_fc_store[[paste0(best_arima_nm, " (rolling)")]]),
  SARIMA = as.numeric(ew_fc_store[[paste0(best_sarima_nm, " (rolling)")]]),
  ARIMAX = as.numeric(ew_fc_store[[paste0(best_arimax_nm, " (rolling)")]])
)

pairs(scatter_df, main = "Scatter matrix: Persistence vs best ARIMA/SARIMA/ARIMAX",
      pch = 19, col = "blue", cex = 0.6)
```



Interpretation

- The rolling window (R = 80 quarters) re-estimates model parameters at each step using only the most recent 20 years of data. This prevents observations from old regimes from diluting current parameter estimates, and better tracks structural shifts over the 92-quarter holdout.
- Comparing the rolling window ARIMA/SARIMA/ARIMAX models directly against benchmark rules (Persistence, Average, SES, Holt, HW Additive) reveals whether the added complexity of parametric models is justified.
- If the rolling window RMSE is similar to the fixed-parameter RMSE, this confirms that the series is close to a random walk, and parameter re-estimation adds little value.
- The persistence benchmark remains the key reference: any model must beat persistence to justify its complexity.

ARIMAX observations:

- We evaluate two regressor sets: **Full** (`ldgpdi_lag1`, `ldcpi_lag2`, `ldgpdi_lag3`) includes all CCF-justified lags; **Reduced** (`ldgpdi_lag1` only) retains only the regressor that is statistically significant at the 5% level in the initial window fit.
- If the Reduced ARIMAX matches or beats the Full ARIMAX on rolling-window RMSE, parameter estimation noise from the insignificant regressors is hurting forecasts and the parsimonious specification is preferred.
- The coefficient significance table from the initial window fit provides a first diagnostic: insignificant coefficients suggest the ARIMA residuals are already orthogonal to the regressors. In that case, including them inflates estimation variance and may worsen out-of-sample performance.
- The rolling window is particularly informative for evaluating regressor usefulness: if ARIMAX RMSE tracks closely with or exceeds ARIMA RMSE as the window rolls through different regimes, the exogenous variables should be dropped in favour of the more parsimonious ARIMA specification.